

实验三：跨平台应用开发

一、实验基本信息

- 实验编号：d20301035103、d20301009203
- 学时分配：4 学时
- 实验类型：验证型实验
- 每组人数：6 人（分组协作完成）
- 对应课程目标：课程目标 2

二、实验目的

1. 掌握 **Flutter** 跨平台应用的核心开发流程与技术要点。
2. 掌握 **React Native** 跨平台应用的组件化开发与数据请求方法。
3. 掌握 **uni-app** 多端统一开发模式与页面渲染机制。
4. 掌握 **.NET MAUI** 企业级跨平台应用开发与 **MVVM** 架构。
5. 理解主流跨平台开发框架的技术原理、渲染模式与差异特点。
6. 能够根据项目需求、团队技术栈与性能要求，合理选择跨平台框架。

三、实验环境

3.1 Flutter 开发环境

- 开发工具：Android Studio / VS Code
- 开发语言：Dart
- UI 框架：Material Design
- 状态管理：Provider
- 网络请求：http 官方插件
- 下拉刷新：pull_to_refresh 插件

3.2 React Native 开发环境

- 开发工具：VS Code
- 开发语言：JavaScript / TypeScript
- UI 框架：React Native 原生组件
- 网络请求：Axios
- 下拉刷新：RefreshControl 组件
- 状态管理：函数式 Hooks

3.3 uni-app 开发环境

- 开发工具：HBuilderX
- 开发语言：Vue.js
- UI 框架：uni-app 内置组件
- 状态管理：Pinia（可选扩展）
- 网络请求：uni.request

3.4 .NET MAUI 开发环境

- 开发工具：Visual Studio 2022
- 开发语言：C#
- UI 框架：XAML / MAUI 原生组件
- 状态管理：CommunityToolkit.Mvvm (MVVM)
- 网络请求：HttpClient

四、实验内容

4.1 核心任务

基于四种主流跨平台框架分别实现**业务数据列表模块**，完成以下统一功能：

1. 对接 RESTful API 完成网络数据请求
2. 实现 JSON 数据解析与模型绑定

3. 实现列表下拉刷新功能
4. 加入加载状态、异常状态与错误处理机制

4.2 实战案例

开发“智慧校园”新闻列表模块：从后端接口获取校园新闻数据，完成 JSON 解析并在列表展示，支持下拉刷新、数据更新与异常提示，实现一套逻辑、多端运行的跨平台效果。

数据结构示例

```
json
{
  "code": 200,
  "data": {
    "items": [
      {
        "id": 1,
        "title": "智慧校园更新通知",
        "content": "系统升级通知...",
        "author": "管理员",
        "time": "2024-01-01 10:00:00"
      }
    ]
  }
}
```

五、实验步骤

5.1 Flutter 新闻列表开发

1. **创建项目**：安装 Flutter SDK，执行 `flutter create news\_list` 新建项目。
2. **配置依赖**：在 `pubspec.yaml` 引入 http、provider、pull_to_refresh 库。
3. **构建数据模型**：创建 News 实体类，完成 JSON 序列化。
4. **状态管理**：编写 NewsProvider，统一管理数据与刷新状态。
5. **实现列表**：使用 ListView.builder 渲染列表，结合 SmartRefresher 实现下拉刷新。
6. **异常处理**：增加加载中、请求失败、空数据等状态展示。

5.2 React Native 新闻列表开发

- 1. **环境准备**: 安装 Node.js, 使用 `npx react-native-init` 创建项目。
- 2. **安装依赖**: 引入 axios、refresh-control 等插件。
- 3. **页面开发**: 使用函数组件 + useState/useEffect 管理数据与生命周期。
- 4. **列表渲染**: 通过 FlatList 高效渲染列表。
- 5. **下拉刷新**: 使用 RefreshControl 组件实现刷新逻辑。
- 6. **状态处理**: 完善请求加载、网络异常等交互提示。

5.3 uni-app 新闻列表开发

- 1. **创建项目**: 打开 HBuilderX, 新建 uni-app 项目。
- 2. **页面配置**: 在 `pages.json` 配置路由与导航栏。
- 3. **数据请求**: 使用 `uni.request` 调用接口, 解析返回数据。
- 4. **页面渲染**: 通过 `v-for` 渲染新闻列表。
- 5. **下拉刷新**: 使用 scroll-view 或页面级下拉刷新实现数据更新。
- 6. **多端适配**: 统一样式与交互, 保证小程序、App 端表现一致。

5.4 .NET MAUI 新闻列表开发

- 1. **创建项目**: 打开 Visual Studio 2022, 新建 .NET MAUI 项目。
- 2. **引入工具包**: 安装 CommunityToolkit.Mvvm 支持 MVVM 模式。
- 3. **模型与视图模型**: 编写 NewsItem 模型与 MainViewModel。
- 4. **数据请求**: 使用 HttpClient 实现异步网络请求。
- 5. **页面布局**: 使用 CollectionView 展示列表, RefreshView 实现下拉刷新。
- 6. **绑定与通知**: 通过数据绑定实现页面自动刷新。

六、实验结果

6.1 成功标准验证

成功标准	Flutter	React Native	uni-app	MAUI
------	---------	--------------	---------	------

列表页面功能完整	✓	✓	✓	✓
RESTful API 数据请求正常	✓	✓	✓	✓
JSON 数据解析正确	✓	✓	✓	✓
下拉刷新功能可用	✓	✓	✓	✓
加载/错误状态处理	✓	✓	✓	✓

6.2 各框架界面特点

- Flutter**: 遵循 Material Design 规范，蓝色主题、卡片式布局、圆角阴影、信息层级清晰，跨端视觉高度统一。
- React Native**: 简洁白底、扁平卡片、分割线清晰，贴近原生系统风格，触摸交互流畅。
- uni-app**: 类小程序设计风格，圆角卡片、排版简洁，多端适配性强，开发成本低。
- MAUI**: 原生 Windows 风格，现代化 UI、阴影层次强，适合企业级桌面/移动应用。

6.3 框架对比分析（精简版）

框架	开发语言	渲染方式	核心特点
Flutter	Dart	自绘引擎（Skia）	高性能、UI 统一、跨端一致性高
React Native	JS/TS	JS 桥接 + 原生渲染	生态丰富、前端友好、开发快速
uni-app	Vue	编译/混合渲染	一套代码多端发

			布，支持小程序/ App/H5
MAUI	C#	原生渲染	.NET 生态、MVVM、企业级应用友好

6.4 跨平台应用架构图（文字版）

```
plain text
接口层 → 网络请求层 → 数据解析层 → 状态管理层 → UI 渲染层

Flutter: http → Model → Provider → Widget（自绘）
React Native: Axios → JSON → Hooks → Native 组件
uni-app: uni.request → JSON → Pinia/Vue → 编译渲染
MAUI: HttpClient → Model → MVVM → 原生控件
```

6.4 功能实现截图——MAUI 新闻列表界面

智慧校园新闻

智慧校园系统升级通知

为了提升用户体验，智慧校园系统将于本周五晚上进行升级维护.....

□ 信息中心

□ 2024-01-15 09:00

期末考试安排公布

2023-2024学年第一学期期末考试安排已经公布，请各位同学及...

□ 教务处

□ 2024-01-14 14:30

图书馆开放时间调整

寒假期间图书馆开放时间将进行调整，具体安排如下.....

□ 图书馆

□ 2024-01-13 10:00

校园卡充值优惠活动

本周校园卡充值享受9折优惠，最高优惠50元.....

□ 财务处

□ 2024-01-12 16:00

新学期选课通知

2023-2024学年第二学期选课即将开始，请同学们做好准备.....

□ 教务处

□ 2024-01-11 08:00

七、框架对比分析

7.1 开发语言对比

框架	开发语言	语言特点	学习曲线
Flutter	Dart	强类型、面向对象、支持 AOT/JIT	中等
React Native	JS/TS	动态/强类型、前端生态完善	低
uni-app	Vue.js	渐进式、模板语法、易上手	低
MAUI	C#	强类型、.NET 生态、面向对象	中等

7.2 性能对比

框架	性能评分	渲染方式	内存占用
Flutter	★★★★★	自绘引擎	中等
React Native	★★★★☆	桥接原生渲染	中等
uni-app	★★★☆☆	混合渲染	偏高
MAUI	★★★★☆	原生渲染	中等

7.3 开发效率对比

框架	开发效率	热重载	调试体验
Flutter	高	优秀	优秀

React Native	高	良好	良好
uni-app	极高	优秀	良好
MAUI	中等	一般	优秀

7.4 生态与支持对比

框架	社区活跃度	第三方库	官方支持
Flutter	非常活跃	丰富	Google
React Native	活跃	极丰富	Meta
uni-app	活跃	丰富	DCloud
MAUI	一般	一般	Microsoft

7.5 适用场景分析

- 1. **Flutter**: 高性能跨端 App、强自定义 UI、复杂动画、全平台视觉统一项目。
- 2. **React Native**: 前端团队快速转型、中小型应用、注重原生体验的项目。
- 3. **uni-app**: 需同时发布小程序 + H5 + App、Vue 技术栈、快速交付项目。
- 4. **MAUI**: .NET 技术栈、企业级内部系统、Windows/移动跨平台应用。

八、实验总结

8.1 技术收获

- 1. 掌握 Flutter 基于 Dart + 声明式 UI 的开发模式，理解 Provider 状态管理思路。
- 2. 掌握 React Native 组件化与 Hooks 开发方式，熟悉桥接渲染原理。
- 3. 掌握 uni-app 多端编译、路由配置与网络请求，理解跨端适配逻辑。
- 4. 掌握 MAUI 的 MVVM 架构与 XAML 界面开发，熟悉企业级开发规范。

5. 能够独立完成跨平台列表、网络请求、下拉刷新、异常处理等通用功能。

8.2 项目结构

```
plain text
olapdy_project/
├─ flutter_news_list/      Flutter 项目
├─ react_native_news_list/ React Native 项目
├─ uniapp_news_list/       uni-app 项目
└─ maui_news_list/         .NET MAUI 项目
```

8.3 框架选择建议

- 追求极致性能与 UI 统一：选用 Flutter
- 追求快速开发、前端友好：选用 React Native
- 追求多端发布（小程序+App+H5）：选用 uni-app
- 属于 .NET 技术栈、企业级开发：选用 MAUI

8.4 后续学习建议

1. 深入学习各框架高级特性与性能优化方案。
2. 掌握主流状态管理：Riverpod、Redux、Pinia、MVVM 等。
3. 学习路由管理、权限处理、本地存储与原生能力调用。
4. 学习接口封装、统一异常处理、日志与埋点方案。
5. 了解自动化测试、CI/CD 与应用发布流程。

九、思考与讨论

9.1 跨平台框架优缺点

优点：

- 一套代码多端运行，显著降低开发与维护成本
- 开发效率高，迭代速度快
- 技术栈统一，便于团队协作

缺点:

- 部分场景性能略低于原生开发
- 平台特有功能需额外适配或受限
- 调试复杂度相对更高

9.2 跨平台框架选型思路

1. 明确目标平台、性能要求与功能复杂度
2. 结合团队现有技术栈与学习成本
3. 评估开发周期、预算与长期维护成本
4. 验证框架生态、社区与第三方库支持
5. 开发原型进行技术验证与对比

9.3 跨平台开发趋势

1. 渲染引擎持续优化，性能接近原生
2. 原生能力支持更完善，桥接成本降低
3. 开发工具链更成熟，热重载与调试更高效
4. 跨平台与原生融合开发成为主流方案